

Rapport

Kenza Benmansour

May 15, 2026

1 Introduction

Ce projet a pour objectif la création d'un générateur d'analyse lexicale (Gal). Pour une expression rationnelle donnée, l'outil doit créer l'analyseur lexical qui lui est associé. Le but de ce projet était de concevoir un générateur d'analyseur lexical (GAL) en langage C. Un générateur d'analyseur lexical est un programme qui produit automatiquement un analyseur capable de reconnaître des mots appartenant à un langage défini à partir d'expressions rationnelles. Il lit les caractères, les regroupe en lexèmes et produit des tokens pour l'analyseur syntaxique.

Le projet s'inscrit dans le cadre du cours de compilation. Son objectif principal est d'illustrer concrètement les notions théoriques étudiées durant le semestre : expressions rationnelles, arbres syntaxiques, automates finis non déterministes, automates finis déterministes et génération automatique d'un analyseur lexical.

2 Cahier des charges

Le projet devait réaliser les étapes suivantes :

1. Lecture d'une expression rationnelle.
2. Construction de l'arbre syntaxique abstrait (AST).
3. Construction d'un automate fini non déterministe (NFA).
4. Transformation du NFA en automate fini déterministe (DFA).
5. Minimisation du DFA.
6. Génération automatique d'un analyseur lexical en C.
7. Génération automatique des représentations graphiques.

Le flot global du projet est donc :

Regex vers AST : Une expression rationnelle, regex, est transformée en arbre syntaxique AST. Cet arbre représente la structure logique d'une expression rationnelle et sépare les opérations de concaténation, disjonction et étoile de Kleene.

AST vers NFA : Ensuite, à partir de l'arbre syntaxique AST, on construit un automate fini non déterministe, NFA grâce à l'algorithme de McNaughton-Yama-Thompson. Cet automate permet de reconnaître le langage décrit par l'expression rationnelle.

NFA vers DFA : Le NFA est ensuite transformé en automate fini déterministe (DFA).

Regex $\rightarrow$ AST $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ DFA minimal $\rightarrow$ Analyseur lexical

3 Commandes importantes

- `make` : compile l'ensemble du projet.
- `./projet test.txt` : exécute le générateur à partir d'une expression rationnelle contenue dans `test.txt`.
- `make graphs` : génère automatiquement les représentations graphiques des automates en format png
- `dot -Tpng ast.dot -o ast.png` : convertit les fichiers Graphviz `.dot` en images.
- `gcc -Wall -Wextra -std=c11 generated lexer.c -o lexer generated` : compiler le fichier `lexer` généré.
- `make clean` : supprime les fichiers générés lors de la compilation.

4 Architecture choisie

Le projet a été divisé en modules indépendants décrits ci-dessous.

4.1 Organisation des fichiers

- `main.c` : point d'entrée du programme.
- `parser_regex.c/.h` : lecture et analyse des expressions rationnelles.
- `ast.c/.h` : structures et manipulation de l'arbre syntaxique.
- `nfa.c/.h` : construction du NFA.
- `dfa.c/.h` : détermination.
- `minimize.c/.h` : minimisation.
- `graphviz.c/.h` : génération des fichiers DOT.
- `generator.c/.h` : génération automatique du code C.
- `utils.c/.h` : fonctions auxiliaires.

4.2 Justification des choix

La séparation en modules a été choisie pour plusieurs raisons : cela permet d'éviter des fichiers très volumineux difficiles à maintenir, chaque fichier possède une responsabilité unique, cela facilite les tests indépendants de chaque composant et de déboguage. Enfin, cette organisation respecte davantage les bonnes pratiques en C et améliore la lisibilité générale du projet.

5 Construction de l'AST

L'arbre syntaxique donne la structure hiérarchique de l'expression rationnelle. Les types de nœuds suivants ont été définis : symbole, concaténation, union, étoile de Kleene.

5.1 Choix du parseur

Un parseur récursif descendant a été choisi. Cette solution a été retenue pour plusieurs raisons : simplicité d'implémentation, proximité avec la structure grammaticale, facilité de débogage, meilleure compréhension.

Les priorités des opérateurs ont été respectées :

1. étoile de Kleene ;
2. concaténation ;
3. union.

6 Construction du NFA

Le NFA a été construit à partir de l'algorithme de McNaughton–Yamada–Thompson : il est présenté dans le cours, il fournit une méthode systématique, il produit toujours un automate valide, il génère un unique état initial et un unique état final.

7 Déterminisation : de NFA à DFA

Le DFA a été obtenu par l'algorithme des sous-ensembles. Pour cela, plusieurs opérations ont été nécessaires : calcul de l'épsilon-fermeture, calcul des transitions, création des nouveaux états, détection des états acceptants.

7.1 Choix de représentation

Les ensembles d'états ont été représentés par des tableaux booléens. Cette solution a été retenue car : elle est simple à programmer, les comparaisons sont rapides, elle évite des structures complexes.

8 Minimisation

La minimisation du DFA consiste à regrouper les états qui ont le même comportement afin de construire un automate déterministe minimal. On commence par créer des partitions entre les états acceptants et non acceptants, puis on raffine ces partitions en comparant les transitions de chaque état pour chaque symbole de l'alphabet. Ce processus est répété jusqu'à stabilité, c'est-à-dire jusqu'à ce qu'aucune nouvelle séparation d'états ne soit nécessaire.

9 Génération automatique de l'analyseur lexical

L'objectif final était de produire automatiquement un fichier C contenant l'analyseur lexical. Le fichier généré contient : les états, les transitions, les états acceptants, une fonction de reconnaissance, un programme principal. Le choix d'utiliser des tableaux pour représenter les transitions a été retenu car cela simplifie fortement le code généré.

10 Difficultés rencontrées et opinion

Plusieurs difficultés ont été rencontrées durant le projet.

10.1 Gestion des priorités

La concaténation implicite des expressions rationnelles était une difficulté, il n'y a aucun opérateur entre eux. La solution retenue a consisté à séparer en niveau de priorité dans le parseur l'étoile de Kleen, la concaténation et l'Union.

10.2 Gestion mémoire

J'ai appris que contrairement à Java et Python, Le langage C nécessite une gestion manuelle de la mémoire, il ne possède pas de garbage collector. Le projet contient beaucoup de structures dynamiques donc des fonctions spécifiques de libération ont été ajoutées afin d'éviter les fuites mémoire (les fonctions "free" tel que ast free, dfa free, nfa free etc).

10.3 Débogage

C est un langage que j'ai très rarement utilisé, le débogage a pris beaucoup beaucoup de temps à être réalisé.

Plusieurs difficultés techniques ont été rencontrées pendant le développement : erreurs de linkage, problèmes de modularité du code C, gestion des fichiers générés, erreurs liées au Makefile, et des problèmes plus théoriques concernant la construction correcte des AST et des automates finis. Vers la fin de la conception du projet, la visualisation Graphviz a également été utile pour identifier certaines erreurs structurelles dans les automates générés.

10.4 Opinion personnelle

Mon plus gros problème a été dans la gestion de mon propre temps. En effet, en plus de devoir gérer un travail, et des problèmes de santé quotidiens qui m'handicapent énormément, j'ai du avant de commencer ce projet, apprendre le langage compilé C que je n'avais pas vu depuis des années, apprendre le cours, puis apprendre quels sont les bonnes pratiques en C : la gestion de la mémoire particulière à C, fichiers headers .h, fichiers .o, développement des fichiers en plusieurs modules, quels sont les modules qui doivent figurer et pourquoi etc ... Je suis contente d'en avoir appris autant en si peu de temps, mais cela a pris plus de temps qu'il ne fallait (c'est à dire tout le reste de mon temps) et j'ai du laisser de côté les 7 autres cours ainsi que leurs projets de côté.

11 Travail réalisé et non réalisé

Tous les éléments demandés ont été réalisés, c'est à dire tout depuis la création d'un arbre de syntaxe d'une expression rationnelle jusqu'à l'obtention de l'automate fini déterministe minimal associé à cette expression.

- l'arbre de syntaxe abstraite T de e (construit de la gauche vers la droite comme cela a été vu en cours et en TD)
- l'automate fini non déterministe N associé à T contenant potentiellement des transitions, par l'application de l'algorithme de McNaughton, Yamada et Thompson
- l'automate fini déterministe D associé à N par application de l'algorithme de construction de sa table de transition Dtrans vu en cours
- l'automate fini déterministe minimal D de e.

12 Conclusion

Ce projet m'a permis d'étudier concrètement les différentes étapes de l'analyse lexicale et de relier les concepts théoriques vus en cours à leur implémentation pratique. Il m'a également permis d'apprendre à structurer un projet en langage C, à manipuler des structures de données complexes et à concevoir une architecture modulaire. Même si certaines parties peuvent encore être améliorées, les objectifs principaux ont été atteints. Ce fut long et difficile surtout pour déterminer le nouveau de modularité et l'apprentissage d'un langage compilé comme C.